

Contracts for C++ — Rationale

Document #: P2899R1
Date: 2025-03-14
Project: Programming Language C++
Audience: SG21 (Contracts), EWG, LEWG
Reply-to: Joshua Berne <jberne4@bloomberg.net>
Timur Doumler <mail@timur.audio>
Rostislav Khlebnikov <rkhlebnikov@bloomberg.net>
Andrzej Krzemiński <akrzemi1@gmail.com>

Abstract

SG21 has, throughout the C++26 cycle, meticulously followed the plan set forth in [\[P2695R1\]](#) to achieve the goal of delivering a minimal viable product (MVP) for a Contracts feature in C++26. That product is [\[P2900R14\]](#) — Contracts for C++. This paper is a companion paper to [\[P2900R14\]](#) and aims to provide the history, the reasoning, and some of the motivation behind the decisions made for that proposal.

Contents

1	Introduction	4
2	Overview	4
2.1	What Are Contracts For?	4
2.2	Proposed Features	8
2.3	Features Not Proposed	10
2.4	How to Use Contracts	11
2.5	Prior Art	16
2.6	A Short History of Contracts for C++	17
2.7	Contracts and Safety	24
2.8	Why Will Contracts for C++ Succeed Where Other Efforts Failed?	26
2.9	Implementation and Deployment Experience	27
2.10	Implementation-Defined Behavior	27
3	Proposed Design	29
3.1	Design Principles	29
3.2	Syntax	30
3.2.1	Function Contract Specifiers	33
3.2.2	Assertion Statement	37
3.2.3	Attributes for Contract Assertions	40
3.3	Syntactic Restrictions	42
3.3.1	Multiple Declarations	42
3.3.2	Virtual Functions	45

3.3.3	Defaulted and Deleted Functions	54
3.3.4	Constructors and Destructors	56
3.3.5	Await and Yield Expressions	56
3.3.6	Pointers to Functions and Pointers To Member Functions	57
3.3.7	Function Type Aliases	62
3.3.8	Use of C Variadic Functions Parameters	62
3.4	Semantics	63
3.4.1	Name Lookup and Access Control	63
3.4.2	Implicit const -ness	64
3.4.3	The Result Binding	71
3.4.4	Function Parameters in Postconditions	76
3.4.5	Objects Passed and Returned in Registers	81
3.4.6	Not Part of the Immediate Context	83
3.4.7	Function Template Specializations	83
3.4.8	No Implicit Lambda Captures	83
3.5	Evaluation and Contract-Violation Handling	84
3.5.1	Point of Evaluation	84
3.5.2	Coroutines	86
3.5.3	Observable Checkpoints	88
3.5.4	Evaluation Semantics: <i>ignore</i> , <i>observe</i> , <i>enforce</i> , <i>quick-enforce</i>	89
3.5.5	Selection of Semantics	101
3.5.6	Checking the Contract Predicate	107
3.5.7	Elision, Duplication, and Evaluating in Sequence	110
3.5.8	Predicate Side Effects	114
3.5.9	The Contract-Violation Handler	118
3.5.10	The Contract-Violation Handling Process	120
3.5.11	Mixed Mode	120
3.5.12	Constant Evaluation	122
3.6	Noteworthy Design Consequences	124
3.6.1	Undefined Behavior During Contract Checking	124
3.6.2	Invalid Data Member Access in Constructors and Destructors	130
3.6.3	Friend Declarations Inside Templates	131
3.6.4	Recursive Contract Violations	132
3.6.5	Concurrent Contract Violations	132
3.6.6	Throwing Violation Handlers	133
3.6.7	Differences Between Contract Assertions and the assert Macro	138
3.7	Standard Library API	138
3.7.1	The <contracts> Header	139
3.7.2	Enumerations	140
3.7.3	The Class <code>std::contracts::contract_violation</code>	144
3.7.4	The Function <code>invoke_default_contract_violation_handler</code>	148
3.7.5	Standard Library Contracts	149
3.8	Feature Test Macros	150
4	Errata	150
4.1	A Flaw in Section 3.5.3, “Observable Checkpoints”	150

Revision History

Revision 1

- Updated this paper to coordinate with [\[P2900R14\]](#)
- Added the “Implementation-Defined Behavior” section
- Made various clarifications and corrections and included additional references

Revision 0

- First release of this paper, relative to [\[P2900R13\]](#)

1 Introduction

The Contracts¹ paper, [P2900R14], introduces the basic terminology of the proposed Contracts feature, describes its scope and design, and contains the corresponding proposed wording changes. The paper contains few details on the motivation for the design decisions in that proposal, the discussion of alternative design choices that have been considered, and the history of how the proposal and all its parts came about, yet this information is important context. This companion paper provides those details, which will help explain [P2900R14], especially for anyone seeking to contribute further productive work building on what has been done so far. Other supplementary material for [P2900R14] can also be found here.

In this paper, you will find three primary sections. “[Overview](#)” describes the general motivation for the Contracts proposal [P2900R14], the goals we wish to achieve with the feature set of this proposal, and an overview of relevant prior art in other programming languages and the history of standardizing a Contracts feature for C++. “[Proposed Design](#)” contains, for each corresponding subsection in [P2900R14], a summary of the motivation for the proposed design, a history — as complete as possible — for the design decisions in that section, and citations to the many papers, written by members of WG21 and SG21, that have contributed to this design. Finally, “[Errata](#)” contains corrections for errors in the frontmatter of [P2900R14] that cannot be corrected directly in that paper because it has already been approved by WG21 plenary and, therefore, cannot be revised further.

When a particular paper achieved consensus in SG21 and resulted in that paper’s proposed changes being adopted into [P2900R14] or earlier revisions that preceded it, we will largely defer to that paper for a thorough discussion of the motivation and use cases and a deeper understanding of the intent that led to the current state of our proposal. When these design decisions were discussed and made in SG21 or other relevant WG21 subgroups, we summarize those here, along with polls that were taken and their results as called by the chair at the time.

Throughout this paper, we will consistently use the terminology established in [P2900R14], Section 2 to refer to Contracts-related concepts and entities, even though the many papers and other Contracts-related works that we will be citing might be using differing terminology for the same concepts. Such older terminology will be called out whenever necessary for understanding.

2 Overview

2.1 What Are Contracts For?

Section 2.1 of [P2900R14] contains a formal description of what contracts are, explains the crucial distinction between *plain-language contracts* and *contract assertions*, defines a *Contracts facility* as a language feature that lets the user write such contract assertions, and lists the purpose of different contract assertion *kinds* such as precondition and postcondition assertions. What Section 2.1 of [P2900R14] does *not* do is answer the crucial question of what the proposed Contracts facility is actually *for*.

¹We capitalize “Contracts” or we explicitly say “the Contracts feature” or “the Contracts facility” when we are referring to the feature, and we use lowercase “contracts” when we are referring to contracts themselves (as in “plain-language contracts”). We apply the same convention to Ranges, Profiles, Coroutines, and Concepts.

The primary purpose of the Contracts facility proposed in [P2900R14] is to allow the programmer to express expectations about program correctness at certain points in a C++ program, particularly when calling and returning from functions; optionally verify during program evaluation that those expectations are met to thus *identify program defects*; and flexibly, portably, and scalably manage how identified defects should be mitigated.

The existing `assert` macro in header `<cassert>` is arguably also a Contracts facility, albeit a rudimentary one. It offers no contract-violation handling strategy other than calling `std::abort()` nor any kind of customization of its behavior other than transforming into an empty expression by defining `NDEBUG`. For this reason, many codebases resort to custom assertion macros that offer more options. However, these macros have no language support and thus suffer from a number of limitations. An alternative way of stating the goal of [P2900R14] is that the proposed Contracts facility is meant as a superior alternative to the usage of C `assert` and similar *assertion macros* by removing those limitations.

- Not relying on *macros* to express assertions removes many associated problems such as parsing issues,² ODR issues,³ the risk of bit rot,⁴ and the invisibility of *ignored* assertions to tooling due to the macros being textually removed.
- Being able to place precondition and postcondition assertions on the *declaration* of a function, rather than inside the function body, makes them visible without having to inspect the function definition and thus makes them more utilizable for humans, compilers, and tooling, such as IDEs and static analysis.
- Providing a standard syntax that is shared across all components of a program — and eventually the entire C++ ecosystem — makes writing, reading, and processing contract assertions portable and scalable and is a vast improvement over the status quo in which every library is using their own assertion macros.
- Having a program-wide *contract-violation handler* that can be defined by the user and supplied at *link time* makes the *handling* of contract violations at run time equally portable and scalable.
- Being able to directly refer to the *return object* of a function makes many postconditions expressible that cannot be expressed noninvasively with macros.
- Providing a flexible system of *evaluation semantics* that can be chosen in many different ways enables many more use cases and tools than a system where contract checks are either turned on or turned off with a single per-translation-unit build flag.
- Having a design based on the Contracts Prime Directive ([P2900R14]’s Design Principle 1) means that, unlike with macro-based facilities, adding a contract assertion to a program or

²Macros are inherited from C and are not aware of C++ specific syntax such as braced initialization or template argument lists, leading to parsing errors when the predicate contains expressions like `Widget{1, 2}` or `array<int, 3>`.

³When an inline function in a shared header contains an assertion macro, compiling one translation unit with macros turned on and another translation unit with macros turned off in the same function and then linking those together results in an ODR violation.

⁴When assertion macros are turned off, they are textually removed by the preprocessor, which means that the program will still compile if the predicate contains an undeclared identifier or otherwise broken code — until the macros are eventually turned on again.

enabling the checking of such an assertion should not alter the correctness of that program, thereby preventing “Heisenbugs” where a bug appears or disappears when the developer attempts to track it down using assertions.

When designing a Contracts feature for C++, the main challenge is the large number of different use cases for such a facility, especially in a multiparadigm language such as C++. Any choice to prioritize a particular use case can lead to a particular design that might be less ideal for satisfying some other use case.

This property of Contracts is what makes achieving consensus on a particular design for this feature so difficult. It is also a likely explanation for why, after over twenty years of attempts to standardize Contracts for C++, no such proposal has yet been adopted into the C++ Standard.

When SG21 started its work, i.e., after C++20 was feature-complete and failed to include a Contracts facility, we first focused on identifying use cases for Contracts and determining their priority. [P1995R1] lists 197 different use cases which have been categorized by thirty participants into the three priority levels *Must Have*, *Nice to Have*, and *Not Important*. Follow-up papers for this in-depth study include [P2185R0], which categorizes these use cases into categories such as *Correctness* and *Optimization*, and [P2032R0], which analyzes how well pre-SG21 Contracts proposals satisfy those use cases.

While some of the proposed use cases are very specific, we can identify five broad categories of needs into which all known use cases fall, and we list them here in order of priority.

1. **Documentation:** Express the contracts of functions and other software components in *code*, as opposed to comments or separate documentation, thus making them consumable by human readers, IDEs, and other tooling and facilitating correct use of interfaces while developing a program.
2. **Runtime checking:** Evaluate contract predicates *during program execution* to portably and scalably identify defects.
3. **Static analysis:** Provide contract assertions as additional input for static analysis tools to identify defects from the source code without running the program, e.g., through symbolic evaluation.
4. **Optimization:** Provide contract assertions as input for optimizers that can *assume* the contract predicates to be true, and optimize the program based on this assumption to thus generate faster code.
5. **Verification:** Use contract assertions to guide formal verification and correctness proofs for a given software component.

The above priority ordering is consistent with the one provided by Bjarne Stroustrup in [P1711R0].

Over the years, we have realized that trying to specify a Contracts feature that adequately addresses all five categories above in the initial version that we ship is not a viable strategy. Instead, we decided to ship an MVP (minimal viable product): an initial set of functionality and a platform upon which WG21 can agree and that can be extended later. By definition, an MVP cannot satisfy all known use cases but is designed to satisfy basic use cases from the start and simultaneously to allow satisfying other important use cases via future extensions to the C++ Standard.

I am a great fan of the incremental approach — getting a minimal change in place and then improving it based on feedback. I consider that engineering as opposed to the ideal of getting a change perfect in advance — which I consider naive and at odds with reality.

*Bjarne Stroustrup*⁵

This approach is a tried and tested recipe for successfully delivering powerful new language features to the C++ Standard. For example, C++11 introduced the initial version of `constexpr` with the restriction that a `constexpr` function could contain only a single `return` statement and nothing more. This strategy was limiting but useful to get implementation and usage experience with functions that can be evaluated during constant evaluation. C++14 added more features to `constexpr`, as did C++17 and C++20. Our goal is to follow the same approach with Contracts for C++.

We have thus decided to focus on the first three categories of needs — *documentation*, *runtime checking*, and *static analysis*, in that order of priority — in this initial version of Contracts for C++. However, with a feature well-specified to support these three categories, all the other categories of use cases are enabled as well and can be addressed via vendor extensions and/or future extensions to the C++ Standard.

Documentation is a crucial need: A plain-language contract is no good if the user of a library does not know that it exists and thus cannot follow it. To improve on the current situation where plain-language contracts are described in code comments is relatively low-hanging fruit for a Contracts facility: All that is required is a uniform standard syntax for contract assertions that can be placed on function definitions and parsed by humans, compilers, IDEs (enabling them to display hints about the contract to the developer as they type), and other tooling. [P2900R14] provides such a syntax (see Section 3.2). The significant value gained by providing just that is discussed in more detail in Section 3 of [P3376R0].

While well-documented code can greatly increase a user’s ability to write code with fewer bugs and a standard syntax for such documentation that can be recognized by IDEs and other tooling greatly improves that documentation’s visibility, extensive experience has shown the fairly low limit to the effectiveness of documentation at ensuring that users actually use a library correctly. *Runtime checking* provides a concrete mechanism to take contract assertions and make a program that, when run, can tell us if it is incorrect, resulting in identifying bugs as they happen so they can be fixed and incrementally improving a program toward complete correctness.

The feature set provided in [P2900R14] is, therefore, dominated by features that provide the required flexibility, scalability, and portability for runtime checking of contract assertions across all different kinds of C++ codebases: different *evaluation semantics* covering all important use cases (Section 3.5.4) that can be flexibly selected at compile time, link time, load time, or run time on a per-program, per-translation-unit or even per-assertion basis (Section 3.5.5) as well as a global, replaceable contract-violation handler that enables many different strategies to mitigate contract violations (Section 3.5.9).

Prioritizing the needs of *static analysis* tools when designing a Contracts feature for C++ was first proposed in [N4319], based on previous experience with Microsoft SAL and Code Contracts. While enabling correctness checks at run time is the predominant focus of the proposed Contracts feature, static analysis is another important tool to find program defects. In particular, static analysis helps

⁵Evolution Reflector, 2019-06-02

find such defects earlier in the development process and can identify edge cases that might be missed by tests and rarely seen in production. Static analysis is, therefore, complementary to runtime checking.

Static analysis tools are significantly aided by the Contracts feature proposed in [P2900R14], and this concept is discussed in more detail in [P3376R0] and [P3386R0]. Static-analysis tools are already successfully using assertion macros today for control flow analysis and range analysis among other things (see Section 3 of [P3386R0]). However, because assertion macros cannot be placed on declarations (i.e., the tool needs to inspect the definition, which might be unavailable or too complex to reason about) and are not visible to the tool when they are switched off and thus textually removed and due to the lack of a standard syntax (many projects use custom assertion macros instead of the standard `assert` macro), the usefulness of static-analysis tools is limited. Vendor-specific annotations can partially overcome these limitations but are not portable and have their own limitations. [P2900R14] instantly provides more value for such tools by removing all the above limitations.

The *optimization* use case can provide significant benefit in the form of increased performance, but when used incorrectly, this approach can too easily lead to additional undefined behavior in a C++ program. Prioritizing optimization is, therefore, at odds with the primary goal of the proposed Contracts feature to *help identify program defects* and the general push toward more *safety* in C++ (see Section 2.7). More discussion of this use case can be found below: Section 2.3 provides our motivation for excluding *assumptions* from [P2900R14], and Section 3.5.4 contains a record of how this decision came to be. Both sections contain references to the many relevant papers.

The *verification* use case has been a much-discussed topic during the development of [P2900R14]. [P3362R0] makes the case that using contract assertions for verification and correctness proofs is a desirable direction but requires the contract predicate to be restricted to expressions that can be proven to have no side effects and to avoid certain forms of undefined behavior, such as the *strict contracts* proposed in [P2680R1] and again in [P3285R0]. However, we found that strict contracts seem to be too restrictive, regarding which predicates they allow, to be practically useful for the more important *runtime checking* use case (see Section 4 of [P3386R0]). We have thus decided not to pursue this direction. A more detailed discussion of this tradeoff and a record of how this decision came to be can be found in Section 3.6.1.

2.2 Proposed Features

Although intended as an MVP, [P2900R14] does not necessarily represent the smallest possible feature set that could be meaningfully standardized. For example, [P1607R1] proposed a much smaller feature set, and [P2900R14] is commonly criticized as being too large. However, [P2900R14] is indeed *minimal* in the sense that it represents the smallest possible feature set that, as a whole, could achieve SG21 consensus within the existing WG21 standardization process.

Several features have been added to the Contracts proposal between its first, smaller version [P2388R0] and the current version [P2900R14], notably user-defined contract-violation handlers, the *observe* and *quick-enforce* semantics, support for `pre` and `post` on coroutines, and implicit `const`-ness of entities declared outside a contract predicate (informally called “`const`-ification”). Each of these additions was necessary to overcome limitations that, at least for some members of SG21, would have made the entire proposal unviable and therefore unable to achieve the necessary consensus to

proceed toward standardization. In this section, we provide a brief motivation for the scope of the feature set as it is in [P2900R14] today.

[P2900R14] proposes three kinds of contract assertions: *precondition assertions*, *postcondition assertions*, and *assertion statements*.

The purpose of precondition assertions is to identify contract violations when *calling* a function: For the caller, they can act as verification that they called the function correctly, and for the callee, they can establish conditions on which the implementation of the function can rely. Precondition assertions are a key feature of [P2900R14] because they provide the syntax necessary to express postconditions on function *declarations* rather than being limited to function *definitions*, which, as we have seen above, is an essential part of the value that [P2900R14] provides over existing assertion facilities.

The purpose of postcondition assertions is a mirror image of the above: For the caller, they can establish conditions that the code following the function call can rely on, and for the callee, they can act as verification that the function has been implemented correctly. However, postcondition assertions have a less favorable cost-to-benefit ratio than precondition assertions. They are significantly harder to implement (see [P3460R0]), while at the same time we expect them to be less widely used; in fact, much of their utility can also be obtained with a good unit test coverage. SG21, therefore, considered shipping a Contracts facility without postcondition assertions.

SG21, Teleconference, 2021-12-14, Poll

Postconditions should be in the MVP at this time.

SF	F	N	A	SA
1	7	3	4	1

Result: Marginal consensus (if at all)

Despite the marginal consensus in the poll above, the Contracts proposal proceeded *with* postcondition assertions and later gained strong SG21 consensus; even those members generally not in favor of postconditions believe at this point that consensus will not be achieved on a feature without them. After several issues with odr-using nonreference parameters in postcondition assertions came up ([P3484R1], [P3487R0], [P3489R0]), SG21 once again considered removing postcondition assertions from the Contracts proposal, but had strong consensus not to do so (for context of this poll, see Section 3.4.5) and instead to approve targeted fixes for the known issues.

SG21, Teleconference, 2024-11-07, Poll 1

For P2900, remove postconditions, as proposed in P3487R0 Option R1.

SF	F	N	A	SA
0	0	3	9	7

Result: Consensus against

Assertion statements are a straightforward and powerful tool to introduce correctness checks at arbitrary points during the evaluation of a program, either wherever a statement can be placed or, when embedded within an immediately invoked lambda, as parts of an arbitrary expression. By